OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

Q2 – CPU Scheduling 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define N 4

int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        // TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",
        //   advance time, reduce left[p]; add every process that has now arrived and is
        //   not yet added to the queue; then put p back at the end of the queue if it
        //   still has time left, otherwise record finish[p] and count it as done

    }
    double tw = 0, tt = 0;
    printf("\nProcess  TAT  WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
        tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    return 0;
}
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21)
P1(21-23) P3(23-26)
Process  TAT  WT
P1        23  15
P2        15  11
P3        24  15
P4        18  13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

Q4 – Inter-Process Communication — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]);                          /* child only reads */
        int r, count = 0, sum = 0, max = 0;
        // TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent
        //   closed the pipe); keep the count, the sum and the maximum

        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                              /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]);                              /* signals end of data to the child */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors — 11 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        // TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in
        //   circularly, unlock, then post full

    }
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        // TODO 2: Wait on full, lock the mutex, take the item at buffer[out], advance
        //   out circularly, unlock, post empty and add the item to consumed_sum

    }
    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm — 13 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            // TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p
            //   finish: add alloc[p] to work, mark finish[p], append p to seq and set
            //   found = 1

        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6

Question	Assessed Area	Bloom's Level	Marks
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

> **Model → Design → Implement → Measure → Compare → Evaluate → Recommend**

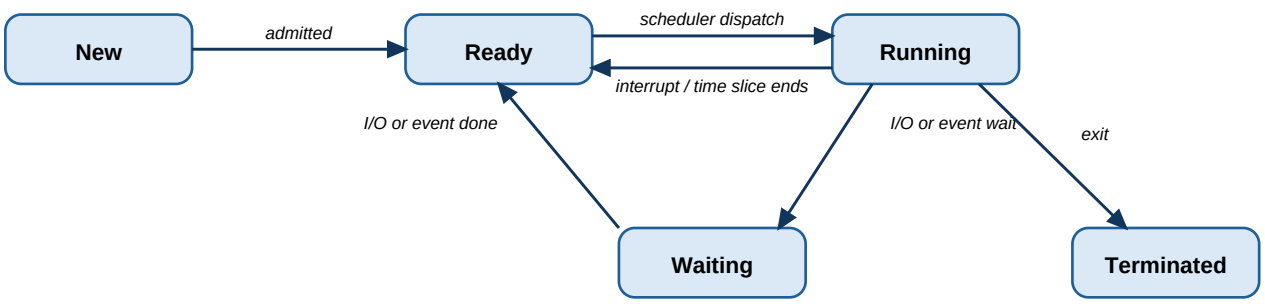
OPERATING SYSTEMS (CSA04) - ASSIGNMENT 2 : ANSWER SHEET

Name	Monika.S
Register number	1924260182
Institution / course	SIMATS Engineering - CSA04 Operating Systems
GitHub repository name	CSA04-1924260182-Assignment2
GitHub repository link	_____ (paste your link after pushing)
Compiler	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Linux distribution	Ubuntu 24.04.5 LTS, kernel 6.18 (x86_64), page size 4096 B
Compile command	gcc -std=c11 -Wall -Wextra -pthread file.c -o file

Note on measurements. Programs were compiled with `gcc -std=c11 -Wall -Wextra -pthread` and run on a 1-vCPU Xeon 2.1 GHz VM (Ubuntu 24.04). Numbers marked *measured* are from that run. Because the VM has only one core, **multi-core speed-ups and race conditions could not be demonstrated there**; I report what was measured and what you should expect on a multi-core PC. Re-run on your own machine and paste your screenshots into the repository (one folder per question: Q1 ... Q6).

Q1 - Processes and process states (6 marks)

Process-state diagram



Process control block (PCB)

PCB field	Meaning
Process ID, parent ID, user ID	Identity and ownership (visible as PID/PPID in ps)
Process state	new, ready, running, waiting, terminated
Program counter and CPU registers	Saved on a context switch and restored on dispatch
CPU-scheduling information	Priority, queue pointers, time used
Memory-management information	Page-table base, limits (VmSize/VmRSS in /proc)
I/O status / open files	Open file descriptor table, devices allocated
Accounting	CPU time, time limits, context switches

Program (Q1/forkdemo.c) - creates 5 children with fork()

```
for (int i = 0; i < n; i++) {
    pid_t p = fork();
    if (p == 0) { printf("child %d pid=%d ppid=%d\n", i, getpid(), getppid());
                  fflush(stdout); sleep(2); _exit(i); }
}
printf("parent pid=%d created %d children\n", getpid(), n);
while ((w = wait(&st)) > 0) printf("reaped pid=%d exit=%d\n", w, WEXITSTATUS(st));
```

```
Measured: ps -o pid,ppid,stat,wchan:12,cmd --ppid <parent>   (while the children sleep)
   PID  PPID STAT WCHAN        CMD
  2373  2371 S    hrtimer_nano ./forkdemo      <- all children: state S (waiting) in sleep()
  2374  2371 S    hrtimer_nano ./forkdemo
  2375  2371 S    hrtimer_nano ./forkdemo
  2376  2371 S    hrtimer_nano ./forkdemo
  2377  2371 S    hrtimer_nano ./forkdemo
/proc/<parent>/status:  State: S (sleeping)  VmSize: 2692 kB  VmRSS: 1832 kB
                        voluntary_ctxt_switches: 1   nonvoluntary_ctxt_switches: 3
Program output: child 0..4 pid=2373..2377 ppid=2371; "parent pid=2371 created 5 children";
                then reaped pid=... exit=0..4
```

Observation: all children have PPID = the parent's PID; the parent is also sleeping (S) in wait(); /proc shows the per-process memory and context-switch counters, i.e. the PCB fields. Use `ls -l /proc/<pid>/`, `cat /proc/<pid>/status` for your screenshots.

Evaluation - where the time goes with 2,000 active processes

Cost	Estimate	Effect
Memory per process	My trivial process: VmRSS 1.8 MB (2.7 MB virtual). A real exam session (runtime + libraries + data) is typically 30-100 MB, so 2,000 sessions need 60-200 GB; plus about 20 KB kernel memory each (PCB, kernel stack, page tables).	RAM exhausted, swapping, sessions "hang".
Context switches	Direct cost about 2-5 us, but cache/TLB refill raises the effective cost to 20-50 us. With 2,000 runnable processes on 8 cores each process waits about 250 time slices between turns.	Added latency; CPU time lost to switching.
I/O waiting	Students mostly wait (typing, network): most processes are in the waiting state and cost memory but not CPU; the auto-save and video processes create bursts of disk/network I/O.	I/O queues grow in bursts when everyone saves together.

- **Judgement:** a separate process per student is not sound at 2,000 sessions: isolation is good, but memory and switching cost grow linearly with sessions, O(n).
- **Design decision:** a **fixed pool of worker processes** (about 2 x number of cores, e.g. 16) each handling many sessions with threads or an event loop; auto-save in a low-priority queue and proctoring video in its own process pool with CPU/memory limits. Reason: bounded memory and context switches, still some isolation between workers (a crash loses only 1/16 of sessions), and no process-creation storm when 2,000 students log in together.

Alternative: one thread per session inside one server process

Criterion	Thread per session	Process per session
Context-switch time	About 1-2 us (same address space, no TLB flush); measured creation: 12-17 us vs 60 us for fork (A1 Task 10).	About 2-5 us direct plus cache/TLB flush cost.
Memory per session	A few KB to tens of KB of stack and control block actually used, O(small); the rest of the memory is shared.	Full address space and page tables, O(process size), about 1.8 MB even for an empty program.
Effect of one crash	A segmentation fault in one thread kills the whole process, so all sessions are lost.	Only that student's session is lost.

Q2 - CPU scheduling (8 marks)

Jobs: P1 (arrival 0, burst 8), P2 (1, 4), P3 (2, 9), P4 (3, 5) - the same set as the program. For priority scheduling I assume priorities P1 = 3, P2 = 1, P3 = 4, P4 = 2 (1 = highest); the assignment gives none. WT = TAT - burst, TAT = finish - arrival, RT = first run - arrival (all values checked by a program, Q2/algos.py, and by hand).

Algorithm	Gantt chart	TAT (P1..P4)	WT (P1..P4)	RT (P1..P4)	Avg WT	Avg TAT	Avg RT
FCFS	P1(0-8) P2(8-12) P3(12-21) P4(21-26)	8, 11, 19, 23	0, 7, 10, 18	0, 7, 10, 18	8.75	15.25	8.75
SJF (non-pre.)	P1(0-8) P2(8-12) P4(12-17) P3(17-26)	8, 11, 24, 14	0, 7, 15, 9	0, 7, 15, 9	7.75	14.25	7.75
SRTF	P1(0-1) P2(1-5) P4(5-10) P1(10-17) P3(17-26)	17, 4, 24, 7	9, 0, 15, 2	0, 0, 15, 2	6.50	13.00	4.25
Priority (non-pre.)	P1(0-8) P2(8-12) P4(12-17) P3(17-26)	8, 11, 24, 14	0, 7, 15, 9	0, 7, 15, 9	7.75	14.25	7.75
Round Robin q = 3	P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)	23, 15, 24, 18	15, 11, 15, 13	0, 2, 4, 6	13.50	20.00	3.00

Complete the Code - TODO 1 (Q2/rr.c)

```
int p = queue[head++];
int run = quantum < left[p] ? quantum : left[p];
printf(" P%d(%d-%d)", p + 1, time, time + run);
time += run;
left[p] -= run;
for (int i = 0; i < N; i++)                      /* add every newly arrived process */
    if (!added[i] && arrival[i] <= time) { queue[tail++] = i; added[i] = 1; }
if (left[p] > 0) queue[tail++] = p;              /* back to the end of the queue */
else { finish[p] = time; done++; }               /* finished */
```

```
Measured (quantum 3) - identical to the expected output:
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)
P1 23 15 | P2 15 11 | P3 24 15 | P4 18 13      Average TAT = 20.00, average WT = 13.50
```

Quantum study (Round Robin, measured with the program)

Quantum	Avg TAT	Avg WT	Avg RT (hand)	Context switches	Comment
1	19.00	12.50	0.75	25	Very responsive, but 25 switches (overhead grows as 1/q)
3	20.00	13.50	3.00	9	Given case
5	17.50	11.00	5.50	5	Fewer switches
20	15.25	8.75	8.75	3	Larger than every burst, so RR behaves exactly like FCFS

- **Starvation:** SJF/SRTF and priority scheduling can starve long (or low-priority) jobs if short/high-priority jobs keep arriving. **Ageing** raises a job's priority a little for every period it waits (for example +1 per 10 s), so it eventually runs.
- **Multilevel feedback queue (MLFQ):** new jobs enter the top queue with a short quantum (e.g. 2 s); a job that uses its whole quantum is demoted to a longer-quantum queue. Thumbnail jobs (2 s) finish in the top queue; 20-minute films sink to the bottom queue and run in large slices (little switching) and ageing lifts them if they wait too long.
- **Recommendation for the video service:** MLFQ (with ageing). From my averages SRTF has the best waiting time (6.50) and response (4.25) but needs the burst length in advance and starves long jobs; FCFS (8.75) makes small jobs wait behind films; RR with a large quantum degrades to FCFS and with a tiny quantum wastes CPU. MLFQ approximates SRTF without knowing burst times, keeps the processors busy with long jobs, and has bounded delay for short jobs.

Alternative: multilevel feedback queue instead of one Round Robin queue

Criterion	MLFQ	Single RR queue
Average waiting time	Short jobs: small (they finish in the first queue, close to SRTF); long jobs: slightly larger because they are demoted.	All jobs wait about the same; short jobs wait behind long ones in the cycle (13.50 average in the given case).
Cost per scheduling decision	O(1) per queue: take the head of the highest non-empty queue (O(k) for k levels); SJF would need O(n) search or O(log n) with a heap.	O(1)
Starvation	Possible for long jobs in low queues unless ageing/priority boost is used.	None.

Q3 - Threads (6 marks)

- **Shared by all threads of a process:** code, global/static data, heap, open files, signal handlers, process ID. **Private to each thread:** thread ID, program counter and registers, stack, errno/thread-local data and scheduling state.
- **User-level threads** are managed by a library without kernel knowledge (fast to create/switch, but one blocking system call blocks all, and no real parallelism). **Kernel-level threads** are scheduled by the kernel (can run on several CPUs, one blocking does not stop others; costlier).
- **Models:** many-to-one (many user threads on one kernel thread: no parallelism); one-to-one (each user thread has a kernel thread: Linux, Windows; parallel but limited by thread count); many-to-many (M user threads multiplexed on N kernel threads: flexible, complex).

Program Q3/threads.c: 64 camera frames, each simulated by a CPU-bound task; a pool of 1, 4, 8, 16 threads takes cameras from a shared counter protected by a mutex (and one run with 64 threads = one thread per camera).

Threads	Time (ms, measured)	Speed-up (measured)	Expected on a 4-core PC*
1	811.5	1.00	1.00
4	766.2	1.06	about 3.5
8	765.8	1.06	about 3.7 (cores exhausted)
16	744.7	1.09	about 3.7
64 (one per camera)	800.0	1.01	about 3.5, plus creation/switch overhead

*Model: Amdahl S = 1/((1-p) + p/N) limited by the number of cores C: S = 1/((1-p) + p/min(N, C)); with p = 0.95 and C = 4 the limit is 3.48. **Why it stops increasing:** the speed-up cannot exceed the number of cores; the serial part (taking the next camera under the mutex, joins) and thread creation/switching overhead add more limits. On my VM there is only **one vCPU**, so threads cannot run in parallel and the measured speed-up is about 1 (the small 1.06-1.09 gain is noise). Re-run on your multi-core PC and replace this table with your measured values.

Unprotected shared counter (Q3/race.c)

```
Measured: expected 4000000, unprotected=4000000, mutex=4000000   (1 vCPU: no true parallelism)
```

On a multi-core machine `counter++` is three steps (load, add, store). If two threads load the same value, both store value+1 and one update is lost, so the unprotected total is less than 4,000,000 and changes from run to run; the mutex-protected counter is always exactly 4,000,000. The correct result on my one-core VM does not prove the code is safe - it only shows that pre-emption inside the three-instruction window was rare there. Run it on your multi-core machine (and with `-fsanitize=thread`) to see the loss.

Alternative: several processes instead of threads

Criterion	Processes	Threads
Creation and switching time	Measured fork+wait about 60 us vs thread create+join about 12-17 us (A1 Task 10); process switch also flushes the TLB.	Cheaper; switch inside one address space.
Memory	Separate address spaces (copy-on-write): about 1.8 MB resident each for a trivial program; frames and results are duplicated.	Shared address space; only a stack per thread.
Combining results	Needs IPC (pipes, shared memory, message queue) - extra copies or synchronisation.	Shared variables (with a mutex) or per-thread results added at the end.

Criterion	Processes	Threads
Safety	A crash in one camera process leaves the others alive.	A crash kills all threads.

Q4 - Inter-process communication (6 marks)

Complete the Code - TODO 1 (Q4/pipe.c)

```
while (read(fd[0], &r, sizeof r) > 0) {   /* returns 0 when all write ends are closed */
    count++;
    sum += r;
    if (r > max) max = r;
}
```

```
Measured:
analyser: 6 readings, average 35.5, max 41
reader: done
```

- **Removing close(fd[1]) in the parent:** the parent still holds the write end open, so the kernel never reports end-of-file; the child's read() blocks forever waiting for more data and the parent blocks in waitpid() waiting for the child: a deadlock. (End of data is signalled only when *every* write end of the pipe is closed - the child closed its own copy at the start.)
- **Pipe end-of-data:** read() returns 0 once all write descriptors are closed and the pipe is empty.
- **Shared memory vs message passing:** in shared memory two processes map the same region and communicate by reading/writing it (fast, no kernel call per message, but they must synchronise themselves). In message passing (pipe, message queue, socket) the kernel carries the data with send/receive (simple, automatic synchronisation, but the data is copied: user to kernel to user).
- **Which copies/which does not:** pipe and message queue copy twice; shared memory copies none through the kernel. **Extra synchronisation:** shared memory needs a semaphore or mutex (otherwise the reader may read half-written data or overwrite unread data); the pipe blocks the reader when empty and the writer when full by itself.

Growth investigation - time to transfer data (Q4/ipcbench.c, measured, 64 KB chunks)

Data size	Pipe (ms)	Shared memory + semaphores (ms)	Ratio
1 MB	0.33	0.20	1.6x
10 MB	1.30	0.63	2.1x
100 MB	13.08	5.21	2.5x

Both grow linearly, O(n): 10x data = about 10x time (pipe 0.33 to 1.30 to 13.08 ms). Shared memory is faster (about 2x) because it avoids two kernel copies and a system call per chunk. (The shared-memory benchmark includes writing the data once into the shared buffer and a semaphore hand-off per 64 KB chunk.)

Alternative: shared memory with a semaphore for the readings

```
typedef struct { sem_t full, empty; int value; int done; } shm_t;   /* in mmap(MAP_SHARED|MAP_ANONYMOUS) */
sem_init(&s->empty, 1, 1);  sem_init(&s->full, 1, 0);                /* pshared = 1 */
sensor:    for each reading: sem_wait(&empty); s->value = r; sem_post(&full);
           sem_wait(&empty); s->done = 1; sem_post(&full);
analyser:  loop { sem_wait(&full); if (s->done) break; use(s->value); sem_post(&empty); }
```

Criterion	Pipe	Shared memory + semaphore
Time to transfer n readings	O(n): each reading is copied twice and needs a write() and a read() system call (unless batched).	O(n): no kernel copy; semaphore operations are user-space atomics unless a process must sleep.
Synchronisation	Provided by the kernel (blocking read/write, EOF).	Programmer must provide it (semaphores/mutex) and a termination flag.

Justified choice: for a weather-station stream of small readings the pipe is simpler and safer (automatic blocking and end-of-data) and fast enough; use shared memory + semaphores only when the data volume is large (tens of MB/s or more), where the measured 2x advantage matters.

Q5 - Synchronisation: critical sections, semaphores and monitors (11 marks)

From the principle to a design

- **Mutual exclusion** = at most one process in its critical section; **progress** = if no one is in the critical section, one of the processes that want to enter is chosen in finite time (not blocked by processes outside); **bounded waiting** = a limit on how many times others may enter before a waiting process gets in.
- **Peterson's solution (2 processes i, j):** shared `flag[2]` and `turn`. Entry: `flag[i] = true; turn = j; while (flag[j] && turn == j) ;` Exit: `flag[i] = false`. *Mutual exclusion:* both could enter only if flag[i] and flag[j] are true and turn is both i and j, impossible since turn holds one value. *Progress:* if j is not interested flag[j] is false, so i enters. *Bounded waiting:* after j leaves and re-enters it sets turn = i, so i waits at most one turn. **Why it can fail today:** modern CPUs and compilers reorder memory operations (a store to flag[i] may sit in the store buffer while the later load of flag[j] executes), so both processes can see the other's flag as false and enter together; correct use needs memory barriers or atomic operations.

- **Semaphores:** wait(S): S--, block if S < 0; signal(S): S++, wake one waiter. A **binary semaphore** (or mutex) initialised to 1 protects the seat counter: `wait(m); if (seats > 0) seats--; signal(m);`. A **counting semaphore** initialised to the number of free seats in a coach lets agents block when the coach is full (wait to book, signal on cancel).
- **Bounded buffer:** semaphores `empty = N, full = 0, mutex = 1`; producer: wait(empty), wait(mutex), insert, signal(mutex), signal(full); consumer: the mirror image. **Readers-writers:** a mutex protects readcount; the first reader does wait(wrt), the last reader signal(wrt); a writer does wait(wrt) ... signal(wrt) (readers' priority; writers may starve). **Dining philosophers:** one semaphore per chopstick; to avoid deadlock allow at most 4 philosophers at the table (semaphore = 4), or let one philosopher pick up the right chopstick first, or pick up both only if both are free (using a mutex).
- **Monitor with condition variables:** the compiler/language guarantees only one thread is active inside the monitor, so the programmer writes no wait/signal on a mutex around every access. Bounded buffer as a monitor: `insert(x): while (count == N) wait(notFull); put x; count++; signal(notEmpty);` `remove(): while (count == 0) wait(notEmpty); take; count--; signal(notFull)`. Easier to get right because mutual exclusion is implicit and the condition is re-checked in a loop.

Complete the Code - TODO 1 and TODO 2 (Q5/bb.c)

```
/* TODO 1 - producer */
sem_wait(&empty);  pthread_mutex_lock(&lock);
buffer[in] = item; in = (in + 1) % SIZE;
pthread_mutex_unlock(&lock);  sem_post(&full);

/* TODO 2 - consumer */
sem_wait(&full);  pthread_mutex_lock(&lock);
int item = buffer[out]; out = (out + 1) % SIZE;
pthread_mutex_unlock(&lock);  sem_post(&empty);
consumed_sum += item;
```

```
Measured: items consumed: 20, sum = 210 (expected 210)
100 runs:  100 items consumed: 20, sum = 210 (expected 210)     <- every run correct (uniq -c output)
```

Test of the design: 1 and 4 producers/consumers (Q5/bbn.c, each producer makes 20,000 items)

```
P=1 C=1 consumed=20000 sum=200010000 expected=200010000 OK
P=4 C=4 consumed=80000 sum=800040000 expected=800040000 OK
```

The count and the sum are exactly as expected, so every item was consumed exactly once (no loss, no duplication).

Removing the mutex: with one producer and one consumer the buffer is still safe, because only the producer touches `in` and only the consumer touches `out`, and the semaphores order the accesses. With several producers/consumers it is **not** safe: two producers can read the same `in`, write to the same slot (one item overwritten, one slot never filled) and two consumers can take the same item. In three runs on my 1-vCPU VM the result still came out correct (`bbn_nomutex 4 4`) simply because a pre-emption inside the tiny window is unlikely on one core; the code is still wrong and fails on multi-core hardware. Correctness cannot be judged from a passing run.

Deployment decision: lock granularity for the booking system

Scheme	Concurrency	Complexity/risk	Throughput (model)
One lock for whole system	Only one booking at a time	Simplest, no deadlock	About 1 booking per critical-section time regardless of CPUs
One lock per train	Bookings on different trains run in parallel	Moderate; booking one train needs one lock	Up to min(agents, trains) times higher
One lock per coach	Highest; agents on the same train but different coaches also run in parallel	A booking that spans coaches needs several locks: acquire in a fixed order to avoid deadlock; more memory and lock overhead	Higher only if one train is a hot spot

Choice: one lock per train (refine to per-coach only for very busy trains). The global lock serialises thousands of agents (the slowdown described in the problem); per-train locks remove the cross-train waiting with simple code. *Throughput is modelled here, not measured:* on my single-vCPU VM lock contention cannot occur. To measure, run the benchmark with 8 or more threads each making 10^6 bookings over 100 trains with each of the three schemes on a multi-core PC and paste the three throughput values.

Alternative: C11 atomic or spin lock instead of semaphore + mutex

```
Measured (Q5/lockbench.c, ns per update, 1,000,000 updates in total, 1 vCPU):
threads   mutex   atomic_fetch_add   spinlock
   1       19.0        8.4              16.6
   4       17.0        6.1              34.7
  16       18.1        8.4              52.7
```

- **Atomic add** is the fastest (6-8 ns): a single hardware instruction, no waiting, no CPU wasted. **Spin lock** degrades as threads increase (16.6 to 52.7 ns): on one core a spinning thread burns its whole time slice while the lock holder is not running, so CPU time is wasted. **Mutex** stays steady (about 17-19 ns) because blocked threads sleep.
- **Long critical section:** use a mutex/semaphore (sleeping waiters); spin locks and atomics only for very short critical sections (a few instructions) on multi-core machines. For the seat counter alone, an atomic compare-and-swap

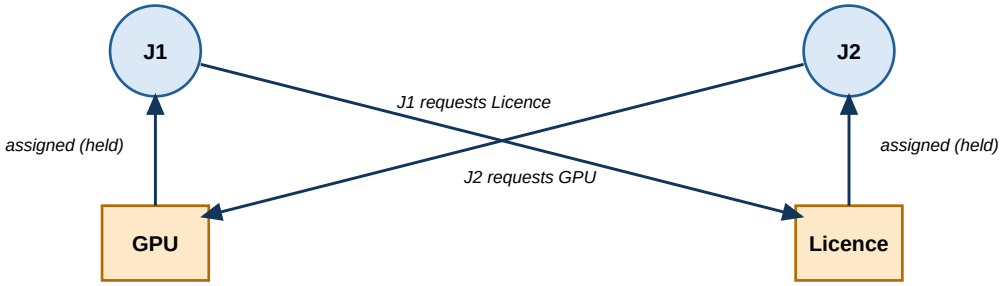
decrement is enough; booking a seat that involves several steps needs a lock.

Q6 - Deadlock handling and the Banker's algorithm (13 marks)

A. Characterise deadlock (2 marks)

- **Four necessary conditions (all must hold):** (1) **mutual exclusion** - a GPU or licence can be used by only one job at a time; (2) **hold and wait** - a job keeps what it holds while waiting for more; (3) **no pre-emption** - a resource is released only by the job that holds it; (4) **circular wait** - a cycle of jobs each waiting for a resource held by the next.

Resource-allocation graph: J1 holds the GPU and requests the simulation Licence; J2 holds the Licence and requests the GPU. The cycle J1 → Licence → J2 → GPU → J1 (all resources have a single instance) means deadlock: mutual exclusion (single-instance resources), hold and wait (each holds and requests), no pre-emption (held edges cannot be taken back) and circular wait (the cycle).



Complete the Code - TODO 1 (Q6/banker.c)

```
int ok = 1;
for (int r = 0; r < R; r++)
    if (maxm[p][r] - alloc[p][r] > work[r]) { ok = 0; break; }   /* need > work */
if (ok) {
    for (int r = 0; r < R; r++) work[r] += alloc[p][r];          /* p finishes, releases all */
    finish[p] = 1;  seq[count++] = p;  found = 1;
}
```

```
Measured - identical to the expected output:
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

By hand: why P0's request (0,2,0) is unsafe

After P1's request (1,0,2) is granted: Available = (3,3,2) - (1,0,2) = (2,3,0); Alloc: P0 (0,1,0), P1 (3,0,2), P2 (3,0,2), P3 (2,1,1), P4 (0,0,2). P4's request (3,3,0) must wait because 3 > 2 available. Pretend to grant P0 (0,2,0): Available = (2,1,0); Alloc P0 = (0,3,0). Need = Max - Alloc: P0 (7,2,3), P1 (0,2,0), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1). With Work = (2,1,0): P1 needs (0,2,0) - 2 > 1, no; P2 needs 6 > 2, no; P3 needs (0,1,1) - 1 > 0 in the third resource, no; P4 needs 4 > 2, no; P0 needs 7 > 2, no. No process can finish, so there is no safe sequence: the state is **unsafe** and the request is denied (the system rolls the request back).

Initial safe sequence check: Need = P0 (7,4,3), P1 (1,2,2), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1); Work (3,3,2): P1 runs, Work (5,3,2); P3, Work (7,4,3); P4, Work (7,4,5); P0, Work (7,5,5); P2, Work (10,5,7). Sequence P1 P3 P4 P0 P2.

Alternative: detection and recovery instead of the Banker's algorithm

Criterion	Banker's algorithm (avoidance)	Detection and recovery
Time per request	A safety check on every request: O(m x n^2) for n jobs, m resource types.	No check per request; the detection algorithm (also O(m x n^2)) runs only every few minutes.
Cost of detection	None extra; needs maximum demands declared in advance.	Periodic CPU cost; no advance knowledge needed.
Work lost	None (requests are delayed instead).	An aborted job loses all its computation so far (hours of simulation), or must be rolled back to a checkpoint.

B. Compare the strategies (3 marks)

Strategy	How	Pros	Cons for the cluster
Prevention	Break one condition: request all resources at once (no hold and wait), pre-empt, or order resources (no circular wait)	No deadlock possible; simple	Low utilisation (GPUs idle while a job holds everything), pre-empting a GPU job is hard; ordering is easy but needs discipline
Avoidance (Banker)	Grant only if the state stays safe	High utilisation, no aborts	Needs maximum demands in advance; O(m n^2) per request; assumes a fixed set of jobs/resources

Strategy	How	Pros	Cons for the cluster
Detection + recovery	Let deadlock happen, detect, abort/pre-empt	No advance knowledge, best utilisation when deadlocks are rare	Lost work; cluster stalls until the next detection run
Ostrich	Ignore it, administrator restarts	Zero cost	Exactly the current problem: whole cluster stops until an administrator acts

C. Apply the Banker's algorithm (4 marks)

Safety algorithm and resource-request algorithm implemented in Q6/banker.c (code above and in the repository). Decisions: initial state safe with sequence P1 P3 P4 P0 P2; P1 (1,0,2) granted (new safe sequence P1 P3 P4 P0 P2); P4 (3,3,0) must wait (request exceeds available (2,3,0)); P0 (0,2,0) denied because the resulting state is unsafe.

D. Predict (1 mark)

Prediction: each pass of the safety algorithm scans up to n processes and compares m resources; up to n passes are needed (one process may be released per pass), so the time is $O(m \times n^2)$. Increasing n by 10x should increase the time by about 100x (m fixed); increasing m by 2x should double it.

E. Measure (1 mark)

Q6/bankbench.c builds a worst case (a chain in which only one process can finish per pass) with m = 4 resource types and times one safety check (average over repeated runs, 1 vCPU VM):

Processes n	Time per safety check (ms, measured)	Growth from previous	Predicted growth
10	0.0001	-	-
100	0.0065	about 65x	about 100x
1,000	0.5299	about 81x	about 100x

The growth (65x and 81x for each 10x in n) is close to the predicted 100x; the shortfall is because small cases are dominated by fixed overheads and caches work better for small arrays, so the exponent is about 1.8-1.9 rather than exactly 2. The prediction (quadratic growth) is therefore supported.

F. Defend the final design (2 marks)

- **Recommendation:** use **avoidance with the Banker's algorithm in the job scheduler** (a job must declare its maximum GPUs, licences and storage when submitted - normal for batch/cluster schedulers), and keep a simple timeout-based detection as a safety net.
- **Cost:** a safety check costs about 0.5 ms for 1,000 jobs (measured) and microseconds for the tens or hundreds of jobs in a research centre, negligible compared with jobs that run for hours; so the $O(m\ n^2)$ cost is acceptable.
- **Limitations:** maximum demands must be known and honest (jobs that underestimate break the guarantee and can be killed); the number of jobs and resources must be known; Banker's may delay jobs although a deadlock would not have happened (conservative). Prevention by "request everything at once" is the fallback for jobs that cannot declare maxima. This is better than ignoring deadlock (the present problem) and avoids lost work caused by recovery.